

**FloodGuard: Cloud Service Integration and Monitoring Through a Serverless and
Microservices Architecture**

Ashok Bhattraï (NP0698[##]), Samir Pokhrel (NP0698[##]),

Anjali Misra (NP0698[##]), and Sangita Tamang (NP0698[##])

School of Computing, Asia Pacific University of Technology and Innovation

CT071-3-3-DDAC: Designing and Developing Cloud Applications

[LECTURER NAME]

[SUBMISSION DATE]

Group G[GROUP NUMBER]

Problem Background 4: Flood Early Warning and Disaster Preparedness

Table of Contents

Introduction.....	3
Task 1 Existing System Summary.....	3
Proposed Cloud Architecture.....	4
Changes Made to the Architecture.....	5
Microservices Design.....	6
Benefits of the Expansion.....	7
System Implementation.....	8
Data Layer.....	9
DynamoDB Weather Snapshot Table.....	9
SQS Work Queues and Dead-Letter Queues.....	9
SNS Topics.....	10
Identity and Access Management.....	11
One Execution Role per Function.....	11
Least-Privilege Policy Verification.....	11
Compute Layer.....	12
Deployment Packaging.....	12
Function Creation.....	13
Event Source Mappings and Concurrency Allocation.....	14
API Layer.....	15
REST API and Resource Tree.....	15
Integrations and Invoke Permissions.....	16
Stage Deployment and Tracing.....	17
Event Sources.....	17
EventBridge Schedule.....	17
S3 Object-Created Notification.....	18
Front Door.....	19
CloudFront Serverless Behaviour.....	19
Microservice Source and Interaction.....	20
System Functionality Post-Implementation.....	21
Results and Discussion.....	22
End-to-End Verification.....	22
Monitoring Coverage.....	23
Measured Performance of the Serverless Tier.....	25
Measured Performance of the Server Tier and Data Stores.....	26
A Concurrency Limit Discovered Through Monitoring.....	26
Distributed Tracing With AWS X-Ray.....	27
Functional Evidence From the Event Pipelines.....	29
Discussion.....	29
Reflection From Each Member.....	30
New Workload Table Matrix.....	32
References.....	34

Introduction

Flooding is among the most destructive recurring natural hazards, and the interval between a rising river and a warned population is often measured in minutes rather than hours. FloodGuard addresses Problem Background 4 by providing a cloud-native early warning and emergency response platform for residents, volunteers, and municipal authorities. Task 1 delivered a working server-based implementation of that platform. Task 2, documented in this report, extends it with a serverless, event-driven tier, refactors the background processing into independently deployable microservices, and adds the monitoring instrumentation needed to evaluate the result.

The report has three objectives. First, it summarises the system inherited from Task 1 and states precisely how the architecture was expanded and why. Second, it documents the implementation of the serverless components and of the microservices that consume Amazon Simple Storage Service (S3), Amazon Simple Notification Service (SNS), and Amazon Simple Queue Service (SQS). Third, it presents monitoring evidence gathered from Amazon CloudWatch and AWS X-Ray and discusses what that evidence reveals about the performance of the expanded system. Every resource described was deployed to a single AWS account in the us-east-1 region and verified live before this report was written.

Task 1 Existing System Summary

Task 1 produced a two-tier server-based deployment. A Next.js frontend and a NestJS application server were each hosted on AWS Elastic Beanstalk, backed by a single Amazon Relational Database Service (RDS) instance running PostgreSQL 16. Three separate Amazon CloudFront distributions fronted the deployment.

All background processing ran inside the application server. Weather polling, flood risk scoring, and alert creation were executed by an in-process scheduled task. The design was functional but structurally constrained in three respects: background work competed with user requests for the same compute resources, the scheduled task could not scale independently of the web workload, and the scheduler would have produced duplicate alerts had the environment ever scaled beyond one instance. Table 1 summarises the inherited components.

Table 1

Components Inherited From the Task 1 Server-Based Deployment

Tier	Service	Configuration	Responsibility
Presentation	Elastic Beanstalk	Next.js 16, single instance	Server-rendered web interface
Application	Elastic Beanstalk	NestJS 11, load balanced	Business logic and REST interface
Data	Amazon RDS	PostgreSQL 16.10, db.t3.micro	Relational persistence
Delivery	Amazon CloudFront	Three distributions	Transport security and caching
Scheduling	In-process cron	Ten-minute interval	Weather polling and alerting

Note. The scheduling row identifies the only component wholly replaced in Task 2. All other rows were retained and extended rather than rebuilt.

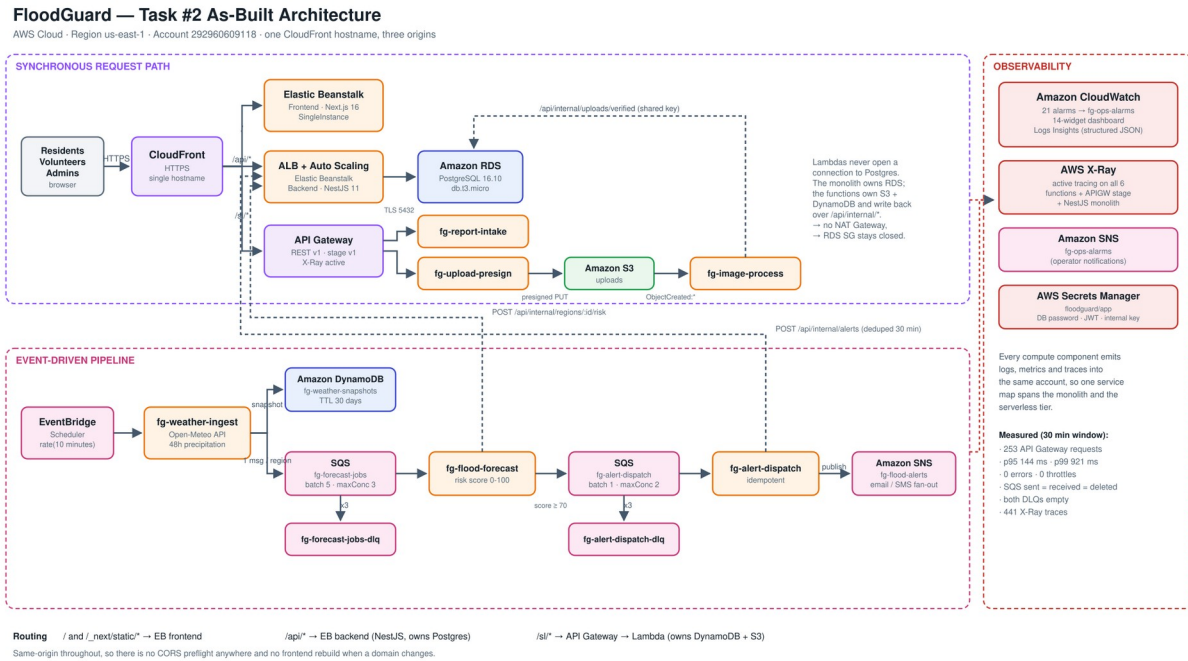
Proposed Cloud Architecture

The architecture proposed for Task 2 introduces a serverless tier alongside the retained server tier, on the principle that workloads with different scaling characteristics should not share a compute boundary. Scheduled ingestion, risk scoring, and notification fan-out are episodic and bursty, whereas the web and application tiers serve steady interactive traffic. Separating them allows each to scale on its own signal, which Newman (2021) identified as the primary operational benefit of service decomposition.

The design consolidates the three CloudFront distributions into one distribution with path-based routing to three origins. It places Amazon API Gateway in front of a set of AWS Lambda functions rather than in front of the existing application server, because only that arrangement makes the functions independently addressable services. It introduces Amazon

DynamoDB for append-only weather time-series data, SQS for decoupling the processing stages, and SNS for alert fan-out. Figure 1 presents the resulting architecture as deployed.

Figure 1
FloodGuard Task 2 As-Built Cloud Architecture



Note. The diagram was generated from the deployed resource inventory rather than from the design proposal, so every component shown corresponds to a provisioned resource. Solid arrows denote synchronous request flow and dashed arrows denote authenticated service-to-service calls.

Changes Made to the Architecture

Reviewing the proposed design against the Task 1 codebase before implementation identified several elements that would not have worked as first drawn. Each was corrected, and those corrections constitute the substantive architectural change between the two tasks. Table 2 records them.

Table 2
Architectural Changes Between Task 1 and Task 2

Element	Issue identified	Resolution
Gateway placement	Positioned in front of Elastic Beanstalk, adding a network hop without adding capability, so no request would ever reach a	API Gateway placed in front of the Lambda functions

Element	Issue identified	Resolution
	function	
Gateway protocol	HTTP APIs emit no X-Ray trace segments, and tracing evidence is required by this task	REST API version 1 selected, which emits trace segments
Trigger chain	EventBridge was drawn as publishing directly to SQS, but a scheduled tick carries no per-item work to distribute	Reordered to EventBridge, then Lambda, then SQS, then Lambda
Queue topology	Work queue and dead-letter queue were drawn as one component, concealing the redrive mechanism	Four queues provisioned: two work queues and two dead-letter queues
S3 integration	S3 was drawn as a passive store with no event wiring	Object-created notification invokes an image verification function
DynamoDB purpose	Intended to hold live risk levels already authoritative in PostgreSQL, creating a dual write with no owner	Repurposed for append-only weather time series with a 30-day expiry
Database exposure	Multi-availability-zone deployment in private subnets exceeded the available budget and required a network address translation gateway	Single availability zone retained, with a two-zone subnet group so promotion remains one API call

Note. Every resolution listed was implemented and verified rather than proposed. The protocol decision recorded in the second row is discussed further under Stage Deployment and Tracing.

Two latent defects in the inherited system were corrected at the same time. The first concerned duplicate alerting. Because the in-process scheduler executes on every running instance, and because the Task 1 configuration constrained instance type but not instance count, the first autoscaling event would have delivered every flood alert twice to real subscribers. Relocating the trigger to Amazon EventBridge Scheduler resolves this, since the scheduler fires once irrespective of fleet size. The second defect concerned credential handling: the Task 1 deployment configuration contained the live database password and token signing secret, which committed both permanently to version control. Credentials are now generated into AWS Secrets Manager and injected at deployment time.

Microservices Design

Six Lambda functions were implemented. Each carries its own least-privilege execution role, an arrangement that matters because the functions do not hold equivalent trust. A single shared role would grant the unauthenticated public intake endpoint permission to publish alerts

to every subscriber, which is a privilege escalation path rather than a convenience. Table 3 states the boundary, trigger, and integration of each service.

Table 3
Microservice Boundaries, Triggers, and AWS Service Integrations

Function	Trigger	Integrates with	Responsibility
fg-weather-ingest	EventBridge, 10 min	DynamoDB, SQS	Retrieve forecasts, store snapshot, enqueue one job per region
fg-flood-forecast	SQS work queue	SQS, internal interface	Compute composite risk score, enqueue alerts above threshold
fg-alert-dispatch	SQS alert queue	SNS, internal interface	Create the alert idempotently, then fan out to subscribers
fg-report-intake	API Gateway	Internal interface	Accept unauthenticated public flood reports
fg-upload-presign	API Gateway	S3	Issue time-limited presigned upload URLs
fg-image-process	S3 object created	S3, internal interface	Verify the file signature and tag the stored object

Note. No function opens a connection to PostgreSQL. The application server owns the relational store and exposes an authenticated internal interface, which preserves a single writer per data store.

Ingestion performs input and output only. Risk scoring is deliberately excluded from it so that a slow change to the scoring algorithm cannot stall data collection. The scoring logic itself was ported from the Task 1 application server without modification, which keeps the two implementations behaviourally identical. Because SQS provides at-least-once delivery, dispatch is idempotent: a matching active alert within a thirty-minute window is suppressed rather than duplicated.

Benefits of the Expansion

The expansion delivers four benefits that the monitoring evidence in the next section quantifies. Independent scalability is the first, since each stage now responds to its own signal: ingestion on schedule, scoring on queue depth, and dispatch on triggered alert volume, rather than by adding whole application instances. Second, the queue boundary confers deployment flexibility, because the scoring function can be replaced while messages accumulate and drain without request loss. Third, resilience improves through explicit dead-letter queues and partial

batch failure reporting, which together prevent one malformed message from forcing redelivery of an entire batch. Fourth, cost efficiency improves because the functions consume compute only while executing, consistent with the economic model described by Adzic and Chatley (2017).

A further benefit concerns data ownership. Keeping the functions outside the virtual private cloud avoids the recurring cost of a network address translation gateway and allows the database security group to remain closed, while the internal interface avoids the dual-write consistency problem that Richardson (2018) identified as a common decomposition failure.

System Implementation

The Task 2 serverless layer was provisioned entirely through the AWS Command Line Interface so that every step is reproducible and auditable rather than a sequence of console gestures. Table 4 lists what was built. The Task 1 server tier, comprising the virtual private cloud, the relational database, both Elastic Beanstalk environments, and the content delivery distribution, was retained and enhanced rather than replaced.

Table 4
Resources Provisioned in the Task 2 Serverless Layer

Service	Resource provisioned	Purpose
DynamoDB	fg-weather-snapshots	Append-only weather time series with 30-day expiry
SQS	Two work queues and two dead-letter queues	Decoupling and failure isolation
SNS	fg-flood-alerts and fg-ops-alarms	Public fan-out and operator alarms
IAM	Six execution roles and one scheduler role	One least-privilege role per function
Lambda	Six functions	The microservices
API Gateway	REST API fg-serverless-api, stage v1	Public entry to two functions
EventBridge	fg-weather-ingest-schedule	Ten-minute ingestion trigger
S3	Object-created notification	Upload is itself the trigger
CloudFront	Path behaviour for /sl/*	Single front door across three origins
CloudWatch	21 alarms and a 14-widget dashboard	Monitoring and response

Note. All commands were executed against a single account in the us-east-1 region and were verified end to end after execution. The account identifier is visible in the resource identifiers shown throughout the figures that follow.

Data Layer

DynamoDB Weather Snapshot Table

The table is partitioned on region and sorted on timestamp, so the latest snapshots for a region resolve in a single query. On-demand billing suits periodic rather than sustained ingestion. A time-to-live attribute expires items after thirty days at no write cost, which is what keeps the table within the free tier.

Figure 2

Creation of the DynamoDB Table and Its Expiry Specification

```
FloodGuard/infra dev ? ) aws dynamodb create-table \
--table-name fg-weather-snapshots \
--attribute-definitions AttributeName=regionId,AttributeType=S AttributeName=ts,AttributeType=S \
--key-schema AttributeName=regionId,KeyType=HASH AttributeName=ts,KeyType=RANGE \
--billing-mode PAY_PER_REQUEST \
--tags Key=Project,Value=$PROJECT \
--query 'TableDescription.{Table:TableName,Status:TableStatus,Keys:KeySchema}'
{
  "Table": "fg-weather-snapshots",
  "Status": "CREATING",
  "Keys": [
    {
      "AttributeName": "regionId",
      "KeyType": "HASH"
    },
    {
      "AttributeName": "ts",
      "KeyType": "RANGE"
    }
  ]
}
FloodGuard/infra dev ? ) aws dynamodb wait table-exists --table-name fg-weather-snapshots && echo ACTIVE
ACTIVE
FloodGuard/infra dev ? ) aws dynamodb update-time-to-live --table-name fg-weather-snapshots \
--time-to-live-specification "Enabled=true,AttributeName=expiresAt" \
--query 'TimeToLiveSpecification'
{
  "Enabled": true,
  "AttributeName": "expiresAt"
}
FloodGuard/infra dev ? )
```

Note. The status moves to ACTIVE with regionId as the partition key, ts as the sort key, and the time-to-live attribute enabled on expiresAt.

SQS Work Queues and Dead-Letter Queues

Four queues were created rather than two. A dead-letter queue is a separate queue reached by a redrive policy, and that policy is the resilience mechanism (Amazon Web Services, 2025a). Visibility timeout was set to six times the longest consumer timeout so that SQS cannot

redeliver a message still being processed, and a maximum receive count of three routes a repeatedly failing message aside instead of looping indefinitely.

Figure 3

Queue Creation and the Redrive Policy Linking Each Work Queue to Its Dead-Letter Queue

```
FloodGuard/infra dev ? } for q in fg-forecast-jobs-dlq fg-alert-dispatch-dlq; do
  aws sqs create-queue --queue-name $q \
    --attributes '{"MessageRetentionPeriod":"1209600"}' \
    --tags "Project=$PROJECT" --query QueueUrl --output text
done
https://sqs.us-east-1.amazonaws.com/292960609118/fg-forecast-jobs-dlq
https://sqs.us-east-1.amazonaws.com/292960609118/fg-alert-dispatch-dlq
FloodGuard/infra dev ? 3s } export FQ_DLQ_ARN=arn:aws:sqs:$REGION:$ACC:fg-forecast-jobs-dlq
FloodGuard/infra dev ? } export AQ_DLQ_ARN=arn:aws:sqs:$REGION:$ACC:fg-alert-dispatch-dlq
FloodGuard/infra dev ? } cat > build/cli/q-forecast.json <<EOF
{"VisibilityTimeout":"360","MessageRetentionPeriod":"345600",
 "RedrivePolicy":{"deadLetterTargetArn":"$FQ_DLQ_ARN","maxReceiveCount\":3\"}}
EOF
FloodGuard/infra dev ? } cat > build/cli/q-alert.json <<EOF
{"VisibilityTimeout":"360","MessageRetentionPeriod":"345600",
 "RedrivePolicy":{"deadLetterTargetArn":"$AQ_DLQ_ARN","maxReceiveCount\":3\"}}
EOF
FloodGuard/infra dev ? }
aws sqs create-queue --queue-name fg-forecast-jobs \
  --attributes file://build/cli/q-forecast.json --tags "Project=$PROJECT" --query QueueUrl --output text
https://sqs.us-east-1.amazonaws.com/292960609118/fg-forecast-jobs
FloodGuard/infra dev ? } aws sqs create-queue --queue-name fg-alert-dispatch \
  --attributes file://build/cli/q-alert.json --tags "Project=$PROJECT" --query QueueUrl --output text
https://sqs.us-east-1.amazonaws.com/292960609118/fg-alert-dispatch
FloodGuard/infra dev ? } aws sqs get-queue-attributes \
  --queue-url https://sqs.$REGION.amazonaws.com/$ACC/fg-forecast-jobs \
  --attribute-names VisibilityTimeout RedrivePolicy --query Attributes
{
  "VisibilityTimeout": "360",
  "RedrivePolicy": "{ \"deadLetterTargetArn\": \"arn:aws:sqs:us-east-1:292960609118:fg-forecast-jobs-dlq\", \"maxR
eceiveCount\":3}"
}
FloodGuard/infra dev ? }
```

SNS Topics

Two topics were kept deliberately separate so that an operational alarm storm can never reach residents, and a public flood alert is never mistaken for an infrastructure notification.

Figure 4

Creation of the Alert and Operations Topics and Confirmation of a Subscription

```
FloodGuard/infra dev ? } aws sns create-topic --name fg-flood-alerts --tags Key=Project,Value=$PROJECT --query T
opicArn --output text
arn:aws:sns:us-east-1:292960609118:fg-flood-alerts
FloodGuard/infra dev ? 2s } Key=Project,Value=$PROJECT --query TopicArn --output text
aws sns create-topic --name fg-ops-alarms --tags Key=Project,Value=$PROJECT --query TopicArn --output text
bash: command not found: --query
arn:aws:sns:us-east-1:292960609118:fg-ops-alarms
FloodGuard/infra dev ? } aws sns create-topic --name fg-ops-alarms --tags Key=Project,Value=$PROJECT --query T
opicArn --output text
arn:aws:sns:us-east-1:292960609118:fg-ops-alarms
```

```
FloodGuard/infra dev ? 130 X aws sns subscribe --topic-arn arn:aws:sns:$REGION:$ACC:fg-flood-alerts --protocol
email --notification-endpoint sameeerpokhrel@gmail.com
{
  "SubscriptionArn": "pending confirmation"
}
FloodGuard/infra dev ? )
```

Identity and Access Management

One Execution Role per Function

Six roles confine the blast radius of a fault in any one handler to that handler alone. Each role trusts only the Lambda service and carries the basic execution and X-Ray write policies in addition to its own inline policy.

Figure 5

Creation of the Six Lambda Execution Roles and Attachment of Their Managed Policies

```
FloodGuard/infra dev ? ) cat > build/cli/lambda-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
"Principal":{"Service":"lambda.amazonaws.com"},"Action":["sts:AssumeRole"]}]}
EOF
FloodGuard/infra dev ? ) for fn in fg-weather-ingest fg-flood-forecast fg-alert-dispatch \
fg-report-intake fg-upload-presign fg-image-process; do
aws iam create-role --role-name ${fn}-role \
--assume-role-policy-document file://build/cli/lambda-trust.json \
--tags Key=Project,Value=$PROJECT --query 'Role.RoleName' --output text
done
fg-weather-ingest-role
fg-flood-forecast-role
fg-alert-dispatch-role
fg-report-intake-role
fg-upload-presign-role
fg-image-process-role
FloodGuard/infra dev ? 9s )

FloodGuard/infra dev ? 9s ) for fn in fg-weather-ingest fg-flood-forecast fg-alert-dispatch \
fg-report-intake fg-upload-presign fg-image-process; do
aws iam attach-role-policy --role-name ${fn}-role \
--policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
aws iam attach-role-policy --role-name ${fn}-role \
--policy-arn arn:aws:iam::aws:policy/AWSXRayDaemonWriteAccess
done
echo attached
attached
FloodGuard/infra dev ? )
```

Least-Privilege Policy Verification

Each role receives an inline policy naming only the resources that its function actually touches. Figure 6 verifies the narrowest of the six.

Figure 6*Evidence of Privilege Separation on the Unauthenticated Intake Function*

```

EOF
for fn in fg-weather-ingest fg-flood-forecast fg-alert-dispatch \
    fg-report-intake fg-upload-presign fg-image-process; do
    aws iam put-role-policy --role-name ${fn}-role --policy-name ${fn}-inline \
        --policy-document file:///dev/pol-${fn}.json && echo "  ${fn} ok"
done
fg-weather-ingest ok
fg-flood-forecast ok
fg-alert-dispatch ok
fg-report-intake ok
fg-upload-presign ok
fg-image-process ok
FloodGuard/infra dev ? 9s > aws iam get-role-policy --role-name fg-report-intake-role \
    --policy-name fg-report-intake-inline --query 'PolicyDocument.Statement[].Action'
[
  [
    "secretsmanager:GetSecretValue"
  ]
]
FloodGuard/infra dev ? )

```

Note. The action list for fg-report-intake-role contains secretsmanager:GetSecretValue and nothing further, which demonstrates that the only unauthenticated endpoint in the system holds no permission to publish to SNS.

Compute Layer

Deployment Packaging

The Node.js 22 runtime already ships the AWS SDK, so five of the six functions package as a handler plus a shared helper of a few kilobytes. Only the presign function declares external dependencies. Small packages mean short initialisation times, which Figure 7 makes visible as a size difference of three orders of magnitude.

Figure 7
Deployment Archive Sizes Across the Six Functions

```
FloodGuard/infra dev ? } D=build/cli
for fn in fg-weather-ingest fg-flood-forecast fg-alert-dispatch \
    fg-report-intake fg-upload-presign fg-image-process; do
    rm -rf $D/pkg-$fn $D/$fn.zip; mkdir -p $D/pkg-$fn
    cp lambdas/$fn/index.mjs $D/pkg-$fn/
    cp lambdas/_shared/common.mjs $D/pkg-$fn/
    if [ -f lambdas/$fn/package.json ]; then
        cp lambdas/$fn/package.json $D/pkg-$fn/
        (cd $D/pkg-$fn && npm install --omit=dev --no-audit --no-fund --silent)
    fi
    (cd $D/pkg-$fn && zip -qr ../$fn.zip .)
    printf " %-20s %s\n" "$fn" "$(du -h $D/$fn.zip | cut -f1)"
done
mkdir: created directory 'build/cli/pkg-fg-weather-ingest'
fg-weather-ingest      4.0K
mkdir: created directory 'build/cli/pkg-fg-flood-forecast'
fg-flood-forecast      4.0K
mkdir: created directory 'build/cli/pkg-fg-alert-dispatch'
fg-alert-dispatch      4.0K
mkdir: created directory 'build/cli/pkg-fg-report-intake'
fg-report-intake       4.0K
mkdir: created directory 'build/cli/pkg-fg-upload-presign'
fg-upload-presign      3.2M
mkdir: created directory 'build/cli/pkg-fg-image-process'
fg-image-process       4.0K
FloodGuard/infra dev ? 3s }
```

Function Creation

Timeouts are set per workload. Ingestion makes ten sequential outbound calls and receives 300 seconds, whereas a function fronted by an endpoint receives 30. Active tracing is enabled on all six, which is what makes the service map in Figure 27 possible.

Figure 8
Creation of the Six Lambda Microservices With Active Tracing Enabled

```
FloodGuard/infra dev ? :> export FQ_URL=https://sqs.$REGION.amazonaws.com/$ACC/fg-forecast-jobs
FloodGuard/infra dev ? :> export AQ_URL=https://sqs.$REGION.amazonaws.com/$ACC/fg-alert-dispatch
FloodGuard/infra dev ? :> export TOPIC=arn:aws:sns:$REGION:$ACC:fg-flood-alerts
FloodGuard/infra dev ? :> create() {
  aws lambda create-function --function-name "$1" \
    --runtime nodejs22.x --architectures arm64 \
    --role arn:aws:iam::$ACC:role/$1-role \
    --handler index.handler --zip-file fileb://build/cli/$1.zip \
    --timeout "$2" --memory-size 512 --environment "Variables=$3" \
    --tracing-config Mode=Active --tags Project=$PROJECT \
    --query "(Fn:FunctionName,Runtime:Runtime,Arch:Architectures[0],TO:Timeout,Tracing:TracingConfig.Mode)"
}
FloodGuard/infra dev ? :> create fg-weather-ingest 300 "${SECRET_NAME=Floodguard/app,BACKEND_URL=$CF_URL,DOB_TABLE=fg-weather-snapshots,FORECAST_QUEUE_URL=$FQ_URL}"
{
  "Fn": "fg-weather-ingest",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 300,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> create fg-flood-forecast 120 "${SECRET_NAME=Floodguard/app,BACKEND_URL=$CF_URL,ALERT_QUEUE_URL=$AQ_URL}"
{
  "Fn": "fg-flood-forecast",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 120,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> create fg-alert-dispatch 60 "${SECRET_NAME=Floodguard/app,BACKEND_URL=$CF_URL,ALERTS_TOPIC_ARN=$TOPIC}"
{
  "Fn": "fg-alert-dispatch",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 60,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> create fg-report-intake 30 "${SECRET_NAME=Floodguard/app,BACKEND_URL=$CF_URL}"
{
  "Fn": "fg-report-intake",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 30,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> create fg-upload-presign 30 "${UPLOADS_BUCKET=$BUCKET}"
{
  "Fn": "fg-upload-presign",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 30,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> create fg-image-process 30 "${SECRET_NAME=Floodguard/app,BACKEND_URL=$CF_URL}"
{
  "Fn": "fg-image-process",
  "Runtime": "nodejs22.x",
  "Arch": "arm64",
  "TO": 30,
  "Tracing": "Active"
}
FloodGuard/infra dev ? :> |
```

Event Source Mappings and Concurrency Allocation

This account permits ten concurrent Lambda executions rather than the default thousand. An uncapped queue consumer scales until it owns all of them and starves the public request path, which API Gateway surfaces to users as a server error. Capping the consumers at three and two therefore reserves five executions for synchronous traffic. Partial batch failure reporting prevents one poison message from forcing redelivery of its healthy batch companions (Amazon Web Services, 2025f).

Figure 9*Queue Consumers Created With Explicit Concurrency Ceilings*

```
FloodGuard/infra dev ? ) aws lambda create-event-source-mapping \
--function-name fg-flood-forecast \
--event-source-arn arn:aws:sqs:$REGION:$ACC:fg-forecast-jobs \
--batch-size 5 --maximum-batching-window-in-seconds 5 \
--scaling-config MaximumConcurrency=3 \
--function-response-types ReportBatchItemFailures \
--query '{Fn:FunctionArn,Batch:BatchSize,MaxConc:ScalingConfig.MaximumConcurrency,State:State}'
{
  "Fn": "arn:aws:lambda:us-east-1:292960609118:function:fg-flood-forecast",
  "Batch": 5,
  "MaxConc": 3,
  "State": "Creating"
}
FloodGuard/infra dev ? ) aws lambda create-event-source-mapping \
--function-name fg-alert-dispatch \
--event-source-arn arn:aws:sqs:$REGION:$ACC:fg-alert-dispatch \
--batch-size 1 --maximum-batching-window-in-seconds 5 \
--scaling-config MaximumConcurrency=2 \
--function-response-types ReportBatchItemFailures \
--query '{Fn:FunctionArn,Batch:BatchSize,MaxConc:ScalingConfig.MaximumConcurrency,State:State}'
{
  "Fn": "arn:aws:lambda:us-east-1:292960609118:function:fg-alert-dispatch",
  "Batch": 1,
  "MaxConc": 2,
  "State": "Creating"
}
FloodGuard/infra dev ? )
```

API Layer*REST API and Resource Tree*

A REST API was selected over an HTTP API for one decisive reason: HTTP APIs do not emit X-Ray trace segments, and tracing evidence is required (Amazon Web Services, 2025d).

Resources are rooted at /sl so that the distribution can forward with an origin path of /v1 and require no path rewriting.

Figure 10
REST API Identifier and Deployed Resource Tree

```
FloodGuard/infra dev ? } export API_ID=$(aws apigateway create-rest-api --name fg-serverless-api \
--description "FloodGuard serverless microservices (Task #2)" \
--endpoint-configuration types=REGIONAL --tags Project=$PROJECT \
--query id --output text)
echo "API_ID=$API_ID"

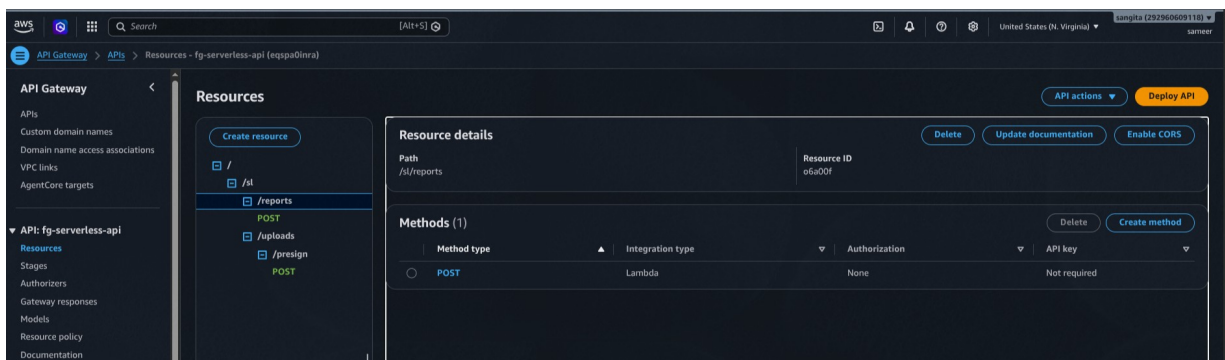
export ROOT=$(aws apigateway get-resources --rest-api-id $API_ID \
--query "items[?path=='/'].id | [0]" --output text)
echo "ROOT=$ROOT"
API_ID=eqspa0inra
ROOT=ob3vytqlie
FloodGuard/infra dev ? } export SL=$(aws apigateway create-resource --rest-api-id $API_ID --parent-id $ROOT --pa
th-part sl --query id --output text)
FloodGuard/infra dev ? } export REPORTS=$(aws apigateway create-resource --rest-api-id $API_ID --parent-id $SL -
-path-part reports --query id --output text)
FloodGuard/infra dev ? } export UPLOADS=$(aws apigateway create-resource --rest-api-id $API_ID --parent-id $SL -
-path-part uploads --query id --output text)
FloodGuard/infra dev ? } export PRESIGN=$(aws apigateway create-resource --rest-api-id $API_ID --parent-id $UPLO
ADS --path-part presign --query id --output text)
FloodGuard/infra dev ? } printf "/sl=%s /sl/reports=%s /sl/uploads=%s /sl/uploads/presign=%s\n" $SL $REPORTS
$UPLOADS $PRESIGN
/sl=m06cli /sl/reports=o6a00f /sl/uploads=z3m5a4 /sl/uploads/presign=zvjx9f
FloodGuard/infra dev ? }
```

Integrations and Invoke Permissions

Lambda proxy integration passes the raw request to the handler, so routing and response shape live in code rather than in gateway mapping templates. The resource-based invoke permission is scoped to this API and this method rather than granted broadly.

Figure 11
Lambda Proxy Integrations and the Resulting Console Resource Tree

```
FloodGuard/infra dev ? } wire() {
  aws apigateway put-method --rest-api-id $API_ID --resource-id $1 \
  --http-method POST --authorization-type NONE --no-api-key-required >/dev/null
  aws apigateway put-integration --rest-api-id $API_ID --resource-id $1 \
  --http-method POST --type AWS_PROXY --integration-http-method POST \
  --uri arn:aws:apigateway:$REGION:lambda:path/2015-03-31/functions/arn:aws:lambda:$REGION:$ACC:function:$2/inv
ocations \
  --timeout-in-millis 29000 >/dev/null
  aws lambda add-permission --function-name $2 --statement-id apigw-$2 \
  --action lambda:InvokeFunction --principal apigateway.amazonaws.com \
  --source-arn "arn:aws:execute-api:$REGION:$ACC:$API_ID/*/POST$3" >/dev/null
  echo " POST $3 -> $2"
}
FloodGuard/infra dev ? } wire $REPORTS fg-report-intake /sl/reports
POST /sl/reports -> fg-report-intake
FloodGuard/infra dev ? 5s } wire $PRESIGN fg-upload-presign /sl/uploads/presign
POST /sl/uploads/presign -> fg-upload-presign
FloodGuard/infra dev ? 4s }
```

Stage Deployment and Tracing

Figure 12

Stage Deployment With X-Ray Tracing Enabled and a Direct Endpoint Test

```
FloodGuard/infra dev ? ) aws apigateway create-deployment --rest-api-id $API_ID --stage-name v1 \
--description "rebuild $(date -u +%Y-%m-%dT%H:%M:%SZ)" \
--query '{Deployment:id,Created:createdDate}' \
{
  "Deployment": "rwrrwb",
  "Created": "2026-08-26T23:10:51+05:45"
}

FloodGuard/infra dev ? 130 * aws apigateway update-stage --rest-api-id $API_ID --stage-name v1 \
--patch-operations \
  op=replace,path=/tracingEnabled,value=true \
  'op=replace,path=/*/metrics/enabled,value=true' \
  'op=replace,path=/*/logging/loglevel,value=INFO' \
--query '{Stage:stageName,Tracing:tracingEnabled}' \
{
  "Stage": "v1",
  "Tracing": true
}

FloodGuard/infra dev ? ) curl -s -o /dev/null -w "presign via execute-api: %{http_code}\n" \
-X POST https://$API_ID.execute-api.$REGION.amazonaws.com/v1/sl/uploads/presign \
-H 'content-type: application/json' -d '{"filename":"a.jpg","contentType":"image/jpeg"}'
presign via execute-api: 200
FloodGuard/infra dev ? 2s )
```

Note. The tracingEnabled setting shown in the second panel is the capability that HTTP APIs lack entirely, and it is the reason the protocol decision in Table 2 was reversed. The third panel is a direct call to the stage endpoint, bypassing the distribution, which isolates the gateway from the front door during testing.

Event Sources

EventBridge Schedule

The schedule replaces the in-process task that ran on every instance. EventBridge fires once regardless of fleet size. Because Identity and Access Management is eventually consistent and the scheduler validates role assumption at creation time, the call is issued inside a retry loop.

Figure 13
Scheduler Role Creation and Verification of the Ten-Minute Trigger

```
FloodGuard/infra dev ? 2s } D=build/cli
cat > $D/scheduler-trust.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"scheduler.amazonaws.com"},"Action":"sts:AssumeRole",
  "Condition":{"StringEquals":{"aws:SourceAccount":"$ACC"}}}]}}
EOF
FloodGuard/infra dev ? } cat > $D/scheduler-invoke.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["lambda:InvokeFunction"],
  "Resource":["arn:aws:lambda:$REGION:$ACC:function:fg-weather-ingest",
    "arn:aws:lambda:$REGION:$ACC:function:fg-weather-ingest:*"]}]}}
EOF
FloodGuard/infra dev ? } aws iam create-role --role-name fg-scheduler-invoke-role \
  --assume-role-policy-document file://$D/scheduler-trust.json \
  --tags Key=Project,Value=$PROJECT --query 'Role.Arn' --output text
arn:aws:iam::292960609118:role/fg-scheduler-invoke-role
FloodGuard/infra dev ? 2s } aws iam put-role-policy --role-name fg-scheduler-invoke-role \
  --policy-name invoke-weather-ingest --policy-document file://$D/scheduler-invoke.json
FloodGuard/infra dev ? } █

FloodGuard/infra dev ? } for i in 1 2 3 4 5 6 7 8; do
  if aws scheduler create-schedule --name fg-weather-ingest-schedule \
    --schedule-expression "rate(10 minutes)" --schedule-expression-timezone UTC \
    --flexible-time-window '{"Mode":"OFF"}' \
    --target file://$D/schedule-target.json --state ENABLED \
    --description "FloodGuard: trigger weather ingestion sweep" >/dev/null 2>&1; then
    echo "created on attempt $i"; break
  fi
  sleep 10
done
created on attempt 1
FloodGuard/infra dev ? } aws scheduler get-schedule --name fg-weather-ingest-schedule \
  --query '{State:State,Expr:ScheduleExpression,Target:Target.Arn}'
{
  "State": "ENABLED",
  "Expr": "rate(10 minutes)",
  "Target": "arn:aws:lambda:us-east-1:292960609118:function:fg-weather-ingest"
}
FloodGuard/infra dev ? } █
```

S3 Object-Created Notification

The invoke permission must exist before the notification is attached, because attaching it validates that the destination is invocable (Amazon Web Services, 2025b). Uploading a photograph is itself the trigger: nothing polls and nothing is scheduled.

Figure 14
Bucket Notification Wiring Uploads to the Image Verification Function

```
FloodGuard/infra dev ? ) aws lambda add-permission --function-name fg-image-process \
  --statement-id s3-invoke-$BUCKET --action lambda:InvokeFunction \
  --principal s3.amazonaws.com --source-arn arn:aws:s3::$BUCKET \
  --source-account $ACC --query Statement
"{\"Sid\": \"s3-invoke-floodguard-uploads-292960609118\", \"Effect\": \"Allow\", \"Principal\": {\"Service\": \"s3.amazonaws.com\"}, \"Action\": \"lambda:InvokeFunction\", \"Resource\": \"arn:aws:lambda:us-east-1:292960609118:function:fg-image-process\", \"Condition\": {\"StringEquals\": {\"AWS:SourceAccount\": \"292960609118\"}, \"ArnLike\": {\"AWS:SourceArn\": \"arn:aws:s3::floodguard-uploads-292960609118\"}}}"
FloodGuard/infra dev ? ) cat > build/cli/s3-notification.json <<EOF
{"LambdaFunctionConfigurations":[{"Id": "invoke-fg-image-process",
  "LambdaFunctionArn": "arn:aws:lambda:$REGION:$ACC:function:fg-image-process",
  "Events": ["s3:ObjectCreated:*"],
  "Filter": {"Key": {"FilterRules": [{"Name": "prefix", "Value": "reports/"}]}}}]}
EOF
FloodGuard/infra dev ? ) aws s3api put-bucket-notification-configuration --bucket $BUCKET \
  --notification-configuration file://build/cli/s3-notification.json
FloodGuard/infra dev ? ) aws s3api get-bucket-notification-configuration --bucket $BUCKET \
  --query 'LambdaFunctionConfigurations[0].{Id:Id,Events:Events,Prefix:Filter.Key.FilterRules[0].Value}'
{
  "Id": "invoke-fg-image-process",
  "Events": [
    "s3:ObjectCreated:*"
  ],
  "Prefix": "reports/"
}
FloodGuard/infra dev ? )
```

Front Door

CloudFront Serverless Behaviour

Adding the `/sl/*` behaviour completes a single hostname serving three origins, so every browser request is same-origin and no cross-origin preflight occurs anywhere in the system.

Figure 15
Distribution Updated With the API Gateway Origin

```
FloodGuard/infra dev ? ) export APIGW_DOMAIN=$API_ID.execute-api.$REGION.amazonaws.com
FloodGuard/infra dev ? ) aws cloudfront get-distribution-config --id $CFID > build/cli/cf.json
FloodGuard/infra dev ? 2s ) export ETAG=$(jq -r '.ETag' build/cli/cf.json)
FloodGuard/infra dev ? ) echo "ETag=$ETAG origin=$APIGW_DOMAIN"
ETag=E1PA6795UKMFR9 origin=eqspa0inra.execute-api.us-east-1.amazonaws.com
FloodGuard/infra dev ? ) jq --arg dom "$APIGW_DOMAIN" --arg stage "/v1" '
  .DistributionConfig as $cfg
  | ($cfg.Origins.Items[] | select(.Id=="eb-backend")) as $otpl
  | ($cfg.CacheBehaviors.Items[] | select(.PathPattern=="/*")) as $btpl
  | $cfg
  | .Origins.Items += [$otpl
    | .Id="apigw-serverless" | .DomainName=$dom | .OriginPath=$stage
    | .CustomOriginConfig.OriginProtocolPolicy="https-only"
    | .CustomOriginConfig.OriginReadTimeout=30]
  | .Origins.Quantity = (.Origins.Items|length)
  | .CacheBehaviors.Items += [$btpl
    | .PathPattern="/*" | .TargetOriginId="apigw-serverless"]
  | .CacheBehaviors.Quantity = (.CacheBehaviors.Items|length)
' build/cli/cf.json > build/cli/cf-new.json
FloodGuard/infra dev ? ) aws cloudfront update-distribution --id $CFID --if-match "$ETAG" \
--distribution-config file://build/cli/cf-new.json \
--query 'Distribution.{Status:Status,Origins:DistributionConfig.Origins.Quantity,Behaviors:DistributionConfig.C
acheBehaviors.Quantity}'
{
  "Status": "InProgress",
  "Origins": 3,
  "Behaviors": 3
}
FloodGuard/infra dev ? 5s )
```

Microservice Source and Interaction

The functions never share a database. The application server owns the relational store, the serverless tier owns DynamoDB and S3, and the two communicate over queues and an authenticated internal interface. Figure 16 shows the source layout: six independent function directories plus one shared helper, each with its own entry point, which is what makes them independently deployable. Ingestion fans work out by sending one message per region in batches of ten, the batch API limit. The forecast consumer collects failed message identifiers into a batch item failure array and returns it, so only the failures are redelivered. Dispatch checks for a matching active alert and returns early before publishing, which is what makes at-least-once delivery safe. Image verification reads the file signature from the object bytes, because a presigned URL pins a declared content type but S3 never checks that the bytes agree with it.

Figure 16*Source Layout of the Six Independently Deployable Microservices*

```
FloodGuard dev ? > find infra/lambda -type f | sort
infra/lambda/fg-alert-dispatch/index.mjs
infra/lambda/fg-flood-forecast/index.mjs
infra/lambda/fg-image-process/index.mjs
infra/lambda/fg-report-intake/index.mjs
infra/lambda/fg-upload-presign/index.mjs
infra/lambda/fg-upload-presign/package.json
infra/lambda/fg-weather-ingest/index.mjs
infra/lambda/_shared/common.mjs
FloodGuard dev ? >
```

System Functionality Post-Implementation

The interfaces below exercise the integrated system end to end, from an unauthenticated visitor through to administrative review. Each was captured from the live deployment reached through the single CloudFront hostname, which confirms that the serverless tier and the retained server tier are reachable as one application.

Figure 17*Public Landing Page Served Through the CloudFront Distribution*

[PASTE SCREENSHOT: the public landing page at the distribution root, with the CloudFront hostname visible in the address bar.]

Figure 18*Resident Dashboard Showing Regional Risk and Active Alerts*

[PASTE SCREENSHOT: the authenticated resident dashboard, showing the risk level and at least one pipeline-generated alert.]

Figure 19*Report Submission Showing the Direct Browser-to-S3 Upload Sequence*

[PASTE SCREENSHOT: the report form with the browser network panel open, showing a request for a presigned URL followed by a separate upload addressed to the S3 hostname. Truncate the signed query string, which is a temporary credential.]

Figure 20*Authority Review of a Submitted Report*

[PASTE SCREENSHOT: a report moving to the verified state, with the attached photograph rendered.]

Figure 21*Alert Issued by an Authority User*

[PASTE SCREENSHOT: the alert creation interface immediately after issuing an alert, with the new alert visible in the active list.]

Figure 22*User Administration and Role Assignment*

[PASTE SCREENSHOT: the user management view with assigned roles. Obscure real electronic mail addresses.]

Results and Discussion

All figures reported in this section were measured on the live deployment using an automated verification and load-generation script. The script exercises every hop of both pipelines and then issues 240 requests across the server and serverless paths in thirty seconds, so that latency percentiles describe a system under load rather than an idle one.

End-to-End Verification

The verification routine proves each hop rather than merely asserting that resources exist, and it discriminates between correct acceptance and correct rejection. All 18 checks passed with no failures. Table 5 summarises the assertions and Figure 23 records the run.

Table 5
End-to-End Verification Assertions and Observed Results

Assertion	Expected	Observed
Front door routes to web, application, and serverless origins	200, 200, 200	200, 200, 200
Internal interface reached without a credential	Rejected	401
Presign endpoint with a valid request and with an executable	200, rejected	200, 415
Public report endpoint with a malformed body and a valid body	Rejected, created	400, 201
Genuine image uploaded, then a text file renamed as an image	verified, rejected	verified, rejected
Scheduled ingestion invoked, failed regions	200, none	200, 0
DynamoDB snapshots written during the run	Increase	20 to 30
Both dead-letter queues after the run	Empty	0 and 0
Duplicate alert message delivered twice	Suppressed	Suppressed
X-Ray traces recorded in the preceding 15 minutes	Non-zero	275

Note. The rejection assertions matter as much as the acceptance assertions. Validation that rejects everything would also pass a test that only checked for failure.

Figure 23
Consolidated End-to-End Verification Result for the Serverless Layer

```
FloodGuard/infra dev 7 45 ) ./deploy.sh 63
✓ account 292940609118 / region us-east-1

63-verify
==> 1. Front door routing (single hostname, three origins)
✓ GET / -> frontend (200)
✓ GET /api/health -> monoLith (200)
✓ GET /api/public/stats (200)
✓ GET /api/docs (swagger) (200)
==> 2. Internal API is not publicly reachable
✓ GET /api/internal/regions unauthenticated (401)
==> 3. Serverless endpoints via /sl/*
✓ POST /sl/uploads/presign (200)
✓ POST /sl/uploads/presign rejects .exe (415)
✓ POST /sl/reports rejects invalid body (400)
✓ POST /sl/reports accepts valid report (201)
==> 4. Upload pipeline end-to-end (presign -> S3 PUT -> event -> sniff -> tag)
✓ real PNG upload tagged verified (verified)
✓ text-as-PNG upload tagged rejected (rejected)
==> 5. Scheduled ingestion pipeline
✓ fg-weather-ingest invocation (200)
✓ ingest had zero failed regions (0)
✓ enqueued 10 region jobs
==> waiting 30s for SQS consumers to drain
✓ DynamoDB snapshots 20 -> 30
==> 6. Dead-letter queues must be empty
✓ fg-forecast-jobs-dlq is empty (0)
✓ fg-alert-dispatch-dlq is empty (0)
==> 7. Alert fan-out with duplicate suppression
✓ duplicate alert suppressed (idempotency holds)
==> 8. Load generation for performance metrics
✓ issuing 120 requests across /api and /sl to populate latency percentiles
✓ 240 requests in 30s (~8 req/s)
==> 9. X-Ray traces recorded
✓ 275 traces in the last 15 min

VERIFICATION SUMMARY
✓ GET / -> frontend 200
✓ GET /api/health -> monoLith 200
✓ GET /api/public/stats 200
✓ GET /api/docs (swagger) 200
✓ GET /api/internal/regions unauthenticated 401
✓ POST /sl/uploads/presign 200
✓ POST /sl/uploads/presign rejects .exe 415
✓ POST /sl/reports rejects invalid body 400
✓ POST /sl/reports accepts valid report 201
✓ real PNG upload tagged verified verified
✓ text-as-PNG upload tagged rejected rejected
✓ fg-weather-ingest invocation 200
✓ ingest had zero failed regions 0
✓ DynamoDB snapshots written 20 -> 30
✓ fg-forecast-jobs-dlq is empty 0
✓ fg-alert-dispatch-dlq is empty 0
✓ duplicate alert suppressed yes
✓ X-Ray traces 275

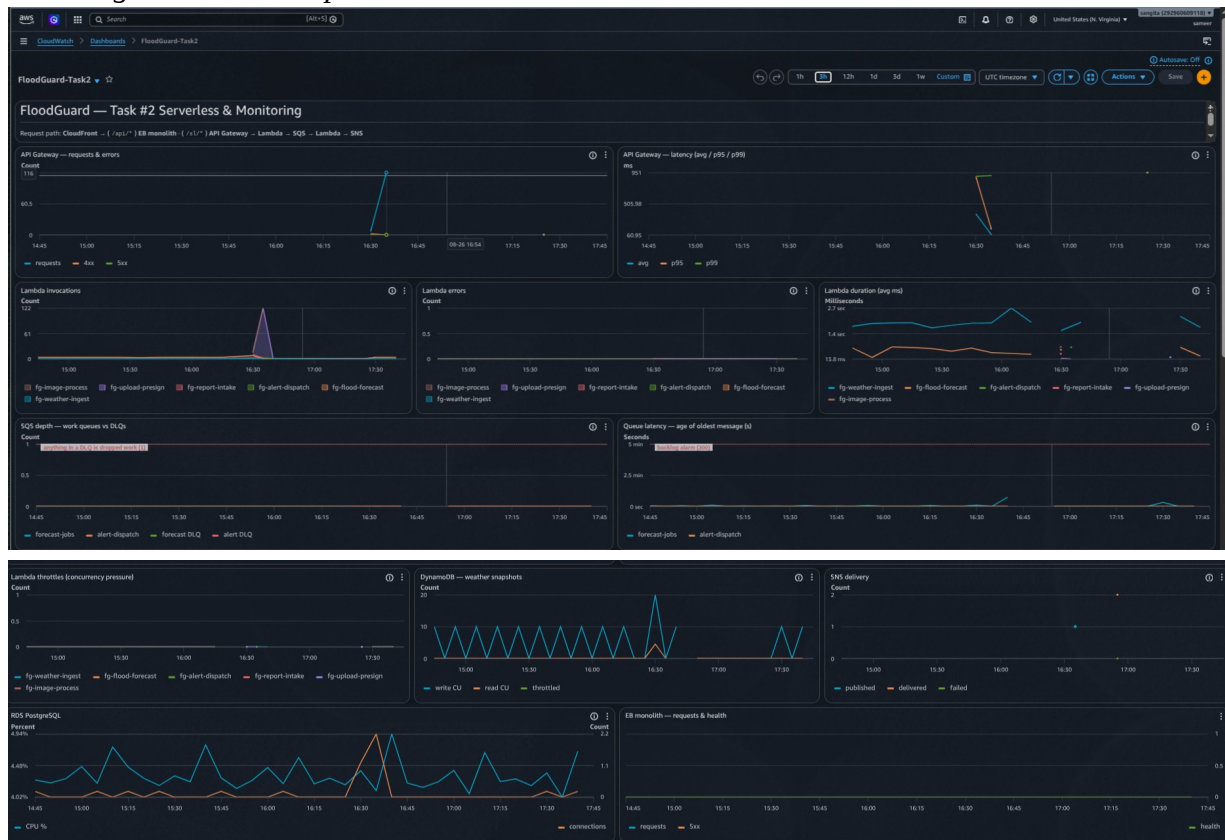
10 passed, 0 failed

✓ dashboard: https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#dashboards/dashboard/FloodGuard-Task2
✓ 63-verify completed in 150s
FloodGuard/infra dev 7 20:37 )
```

Monitoring Coverage

A 14-widget dashboard is laid out along the request path, from front door through the serverless pipeline and queue health to the data tier and the retained server tier, ending in a log query of recent errors across all six functions. All handlers emit single-line structured output so that fields remain queryable.

Figure 24
Monitoring Dashboard Populated Under Load



Twenty-one alarms are configured, every one of them publishing to the operations topic, because a metric without an alarm action is observation without response (Amazon Web Services, 2025e). Three matter most. A non-empty dead-letter queue means work was permanently dropped. An age of oldest message above five minutes is the honest backlog signal, because queue depth can look healthy on a fast-moving queue while consumers fall behind. A Lambda throttle count is a capacity signal rather than a code defect, and the next subsection shows that alarm earning its place.

Figure 25
Alarm Inventory in a Non-Alarming State

```
FloodGuard/infra dev ? > aws cloudwatch describe-alarms --query "MetricAlarms[?starts_with(AlarmName,'fg-')].{
Name:AlarmName,State:StateValue}" --output table
-----
|                               DescribeAlarms                               |
|-----|-----|
| Name                           | State |
|-----|-----|
| fg-alert-dispatch-backlog      | OK    |
| fg-alert-dispatch-dlq-not-empty | OK    |
| fg-alert-dispatch-errors       | OK    |
| fg-alert-dispatch-throttles    | OK    |
| fg-apigw-5xx                   | INSUFFICIENT_DATA |
| fg-apigw-latency-p99           | OK    |
| fg-backend-env-unhealthy       | INSUFFICIENT_DATA |
| fg-flood-forecast-errors       | OK    |
| fg-flood-forecast-throttles    | INSUFFICIENT_DATA |
| fg-forecast-jobs-backlog       | OK    |
| fg-forecast-jobs-dlq-not-empty | OK    |
| fg-image-process-errors        | INSUFFICIENT_DATA |
| fg-image-process-throttles     | OK    |
| fg-rds-cpu-high                | OK    |
| fg-rds-storage-low             | OK    |
| fg-report-intake-errors        | INSUFFICIENT_DATA |
| fg-report-intake-throttles     | OK    |
| fg-upload-presign-errors       | OK    |
| fg-upload-presign-throttles    | OK    |
| fg-weather-ingest-errors       | OK    |
| fg-weather-ingest-throttles    | OK    |
|-----|-----|
FloodGuard/infra dev ? 2s > |
```

Note. Alarms reported as `INSUFFICIENT_DATA` cover error and throttle conditions that had not occurred within the evaluation window, which is the expected state for a healthy system rather than a configuration fault.

Measured Performance of the Serverless Tier

Table 6 reports Lambda metrics across the window covering the load run.

Table 6
Lambda Invocation Counts, Error Counts, and Duration Percentiles Under Load

Function	Invocations	Errors	M (ms)	p95 (ms)	Max (ms)
fg-upload-presign	245	0	13.1	96.2	117.9
fg-report-intake	4	0	277.7	551.0	552.3
fg-flood-forecast	12	0	466.9	968.6	988.5
fg-alert-dispatch	4	0	475.4	765.9	778.4
fg-image-process	2	0	557.5	772.2	778.5
fg-weather-ingest	5	0	1638.3	2194.8	2245.9

Note. M denotes the arithmetic mean duration. Invocation counts differ by function because each is driven by a different event source rather than by uniform traffic.

Two observations follow. First, the gap between the mean of 13.1 ms and the 95th percentile of 96.2 ms for the presign function is initialisation cost. The warm path is a matter of

single-digit milliseconds and the tail is the creation of new execution environments during a burst (Amazon Web Services, 2025c). Provisioned concurrency would remove that tail at a fixed hourly charge, which intermittent academic traffic does not justify. Second, weather ingestion is the slowest function at 1.6 s, but it is intended to be, because it makes ten outbound calls to an external forecast service (Open-Meteo, 2025). That is precisely why scoring was moved behind a queue: a synchronous design would have placed those 1.6 s on a user's request path.

Measured Performance of the Server Tier and Data Stores

Table 7

Server Tier and Data Store Utilisation Across the Measurement Window

Component	Metric	Value
Load balancer	Requests, mean target response time	327, 23.6 ms
Load balancer	Successful responses, server errors	321, 0
RDS PostgreSQL	CPU, connections, free storage	4.3%, 0.4, 17.1 GB
DynamoDB	Items, size, billing mode	3,212, 782 KB, on demand
DynamoDB	Consumed write units, read units	60, 418.5
SNS	Messages published, failed	1, 0

A mean target response time of 23.6 ms against 4.3% database processor utilisation

shows the server tier comfortably under-used. That is the expected outcome of moving polling and scoring off it, because in Task 1 the same instances that served user requests also ran that work in process.

A Concurrency Limit Discovered Through Monitoring

The most useful result of this task came from monitoring rather than from testing, and it changed the deployed configuration.

An early load run returned two server errors out of 253 gateway requests. X-Ray attributed the faults to the gateway stage, and the traces showed a duration of 13 ms with a Lambda segment containing no subsegments and no application error. A genuine handler failure would have produced an error segment, so the request had never reached application code. The

throttle metric for the presign function confirmed two throttles in the same minute, and the account quota supplied the cause: this account permits ten concurrent executions, not the default thousand, and the load generator was issuing bursts of twelve.

The architectural risk this exposed matters more than the failed test. Both queue event source mappings had no concurrency ceiling, so either consumer could occupy all ten slots. A backlog on the forecast queue, which is background work, would therefore starve the presign and intake functions on the public request path. A shared concurrency pool with no allocation lets a background workload hold a user-facing endpoint hostage.

Two changes followed. The queue consumers were capped at three and two, reserving the remaining five executions for synchronous traffic, and the queue absorbs any resulting backlog, which is precisely its purpose. Scaling configuration was chosen over reserved concurrency deliberately, because at an account limit of exactly ten Lambda will not permit reserved concurrency to be allocated at all. The load generator was then bounded below the quota so that the test measures the latency of the system rather than the ceiling of the account. Table 8 records the outcome.

Table 8
Throttling and Server Errors Before and After the Concurrency Allocation

Run	Presign invocations	Throttles	Gateway server errors
Before allocation, burst of 12	118	2	2
After allocation, burst of 8	117	0	0

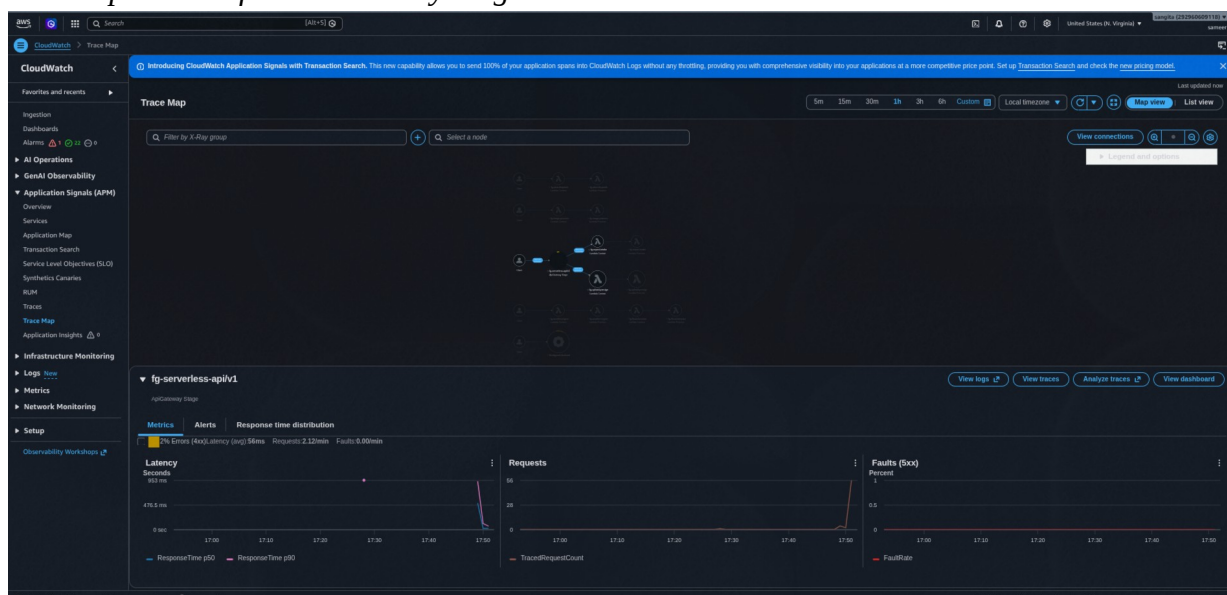
Note. The functional assertions passed in both runs. Only the metrics distinguished a system that worked from a system that would continue to work while a queue was busy.

Distributed Tracing With AWS X-Ray

Active tracing is enabled on all six functions, on the gateway stage, and on the application server. The server required explicit instrumentation, because the platform option enables the trace daemon, which forwards segments but does not create them, so that option

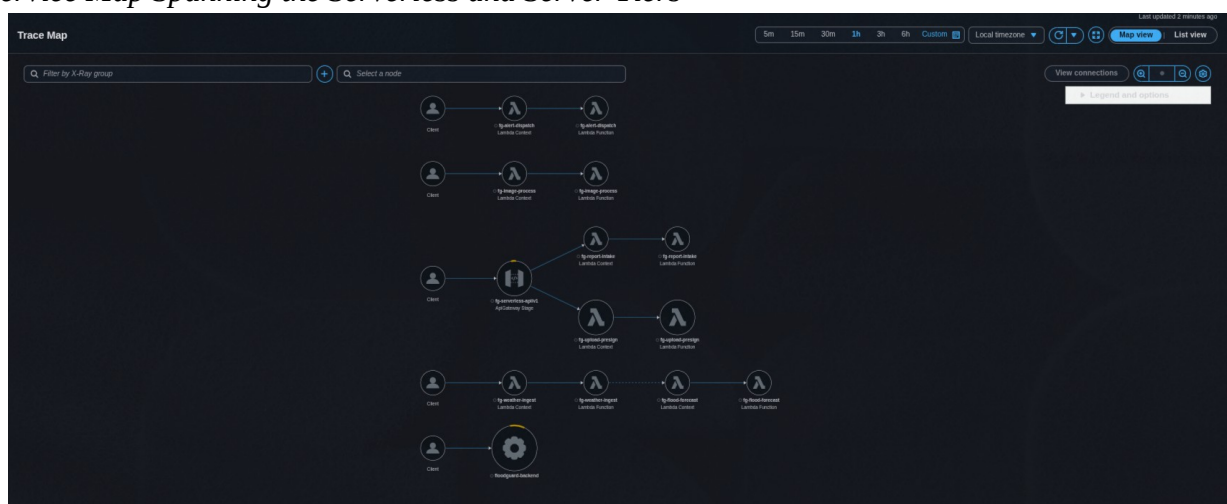
alone yields an empty service map. Because both tiers report into one account, a single map spans the monolith and the serverless tier, which is why the throttling above was attributed to the gateway stage rather than to a handler within minutes of it occurring.

Figure 26
Trace Map Metrics for the Gateway Stage Under Load



Note. Mean latency of 56 ms and a fault rate of zero are recorded against the stage. The 2% client error rate is the deliberate rejection traffic issued by the verification routine, namely the unauthenticated internal call, the malformed report, and the rejected executable upload.

Figure 27
Service Map Spanning the Serverless and Server Tiers



Note. The gateway stage node, the six function nodes, and the application server node appear in one view. The presence of the gateway node is the return on selecting a REST API over an HTTP API, as recorded in Table 2.

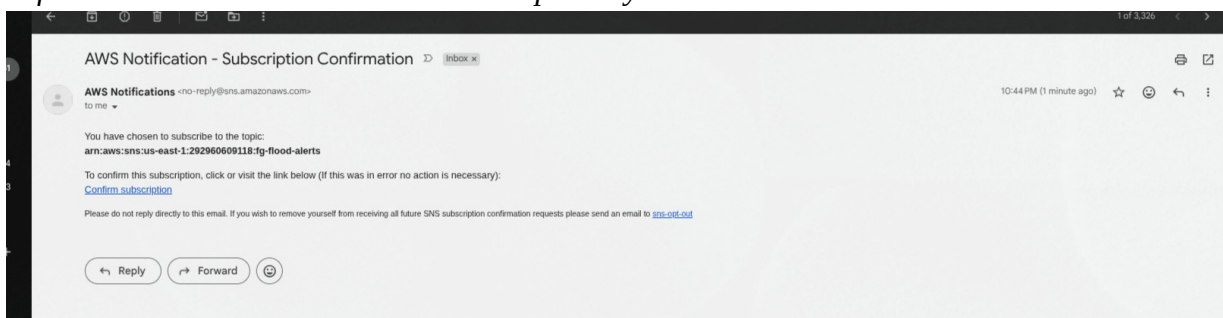
Functional Evidence From the Event Pipelines

Two outcomes confirm that the event wiring fired rather than merely existing. The object tags in Figure 28 were written by the verification function, so their presence proves the object-created notification invoked it and that the file signature check ran on the stored bytes. The message in Figure 29 confirms that fan-out reaches a subscribed endpoint, which is the outcome that matters most in an early warning system.

Figure 28
Object Tags Written by the Image Verification Function

```
FloodGuard/infra dev ? } KEY=$(aws s3api list-objects-v2 --bucket $BUCKET --prefix reports/ \
--query 'reverse(sort_by(Contents,&LastModified)))[0].Key' --output text)
echo "$KEY"
aws s3api get-object-tagging --bucket $BUCKET --key "$KEY"
reports/S17853eb-0076-4d28-92b3-34061f07241e-verify-fake.png
{
  "TagSet": [
    {
      "Key": "detectedType",
      "Value": "unknown"
    },
    {
      "Key": "verification",
      "Value": "rejected"
    }
  ]
}
```

Figure 29
Notification Delivered to a Subscribed Endpoint by Amazon SNS



Discussion

The monitoring evidence supports the four benefits claimed for the expansion. Scalability is evidenced by the server tier utilisation figures in Table 7: each stage now scales on its own

signal, and the offloaded work no longer competes with user requests. Flexibility follows from the queue boundary, which allows the scoring function to be redeployed while messages accumulate and drain, so no request is lost during a deployment. Reliability is evidenced directly, because sent, received, and deleted counts agreed on both work queues with both dead-letter queues empty, so no message was lost and none required redrive. Partial batch failure reporting, the thirty-minute deduplication window, and content verification on upload were each asserted explicitly rather than assumed.

Three limitations deserve statement. The ten-execution quota is a hard ceiling on this account; the allocation described above manages it but does not remove it, and a production deployment would request an increase. The database remains in a single availability zone and is publicly reachable so that migrations can be run from a developer machine, which is acceptable for assessment but not for production, though the two-zone subnet group makes promotion a single API call. Finally, initialisation cost dominates the 99th percentile, which is an inherent trade-off of on-demand serverless compute and one that Jonas et al. (2019) identified as a defining characteristic of the model rather than a defect of this implementation.

Reflection From Each Member

Ashok Bhattraï (M1), weather ingestion and forecasting, S3 and IAM. In Task 1 I built the weather and forecast modules as services running inside the application server, and the hardest part of Task 2 was accepting that a working scheduled task was in fact a design fault. The in-process scheduler runs on every instance, so the moment the environment scaled out, every flood alert would have been delivered twice. That was invisible to us until we tried to move the trigger to EventBridge. The lesson I took from it is that “it works in production today” and “it is correct” are different claims, and only the second survives a change in capacity.

Writing one role per function was more tedious than writing one shared role, but seeing exactly why a shared role would let the unauthenticated report endpoint publish alerts to every subscriber made least privilege concrete rather than a phrase from a lecture. My contribution sat at the front of the pipeline, so everything downstream depended on ingestion staying simple.

Samir Pokhrel (M2), report and upload microservices, API Gateway, CloudFront.

Extracting the presigned upload logic out of the application server into its own function was the point at which microservices stopped being an abstraction for me. My largest realisation came from the front door. In Task 1 we ran three distributions on separate hostnames, which forced cross-origin configuration and a frontend rebuild whenever a domain changed. Collapsing them into one distribution with three path-routed origins removed an entire category of defect rather than fixing individual instances of it. I also learned that a presigned URL is not a security control, because it pins a declared content type while S3 never inspects the bytes, which is why the verification function reads the file signature itself. Testing that by uploading a text file renamed as an image and watching it come back tagged as rejected was the most instructive moment of the project.

Anjali Misra (M3), alert fan-out, SNS, CloudWatch alarms and dashboard. I owned the monitoring requirement and began by treating it as a reporting exercise: build the dashboard, capture the screenshots, submit. What changed my mind was the throttling incident. Every functional assertion passed and the system looked healthy; only the throttle metric and a trace with an empty function segment revealed that a background queue could starve our public interface. I had configured that alarm without really believing it would ever fire. The second lesson was that a metric without an alarm action is only a chart, which is why all 21 alarms publish to a topic. Choosing age of oldest message over queue depth taught me the same thing in

a different form, because depth looks reassuring on a fast queue even while consumers fall behind. Working on alert fan-out also made me careful about who receives what, which is why operational alarms and public alerts use separate topics.

Sangita Tamang (M4), internal service interface, RDS, SQS reliability, X-Ray. My contribution was the boundary between the two tiers, that is the internal interface the functions call instead of connecting to the database directly. Arguing for that constraint was difficult, because a direct connection seemed simpler. What settled it was cost and security together: a function attached to the virtual private cloud needs an address translation gateway with a recurring monthly charge, and a function outside it has no stable address, so the alternative was opening the database port to the public internet. Implementing idempotency taught me the most. Because SQS delivers at least once, I had assumed retries were harmless until I realised that a redelivered message would notify every resident in a region a second time. The deduplication window, and a test that sends the same message twice and asserts that one alert results, is the part of this project I am most confident in.

New Workload Table Matrix

Table 9 records each member's responsibilities across both tasks. Distribution was planned by feature rather than by layer, so that every member owned frontend, backend, and cloud infrastructure work in both tasks rather than specialising into a single tier.

Table 9

Member Responsibilities and Contributions Across Task 1 and Task 2

Member	Task 1 contribution	Task 2 contribution	AWS services owned
M1, Ashok Bhattra	Weather dashboard and interactive risk map; flood forecast module; weather, regions, and forecast backend modules; resident awareness pages	fg-weather-ingest and fg-flood-forecast functions; external forecast integration; DynamoDB time-series schema with 30-day expiry; per-function execution roles; EventBridge schedule	S3, IAM, DynamoDB, EventBridge

Member	Task 1 contribution	Task 2 contribution	AWS services owned
M2, Samir Pokhrel	Flood report system and assistance requests; report, request, and upload modules; presigned upload flow; resident action pages	fg-report-intake, fg-upload-presign, and fg-image-process functions; REST API configuration; single-distribution path routing; file signature verification	API Gateway, CloudFront, VPC, S3 uploads
M3, Anjali Misra	Alert system and region management; authority dashboard; alert module and region administration; review pages	fg-alert-dispatch function; separation of public and operational topics; 21 CloudWatch alarms; 14-widget dashboard; log query views; throttling analysis	SNS, CloudWatch, Elastic Beanstalk, security groups
M4, Sangita Tamang	Volunteer request response; user and authentication system; authentication, user, and evacuation modules; database schema and migrations	Internal service interface and its guard; queue and dead-letter queue topology with redrive policies; partial batch failure handling; alert idempotency; tracing of the application server	RDS, SQS, Secrets Manager, X-Ray

Note. Architecture review, the end-to-end verification routine, this report, and the demonstration were shared activities carried out jointly by all four members.

References

- Adzic, G., & Chatley, R. (2017). Serverless computing: Economic and architectural impact. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (pp. 884–889). Association for Computing Machinery.
<https://doi.org/10.1145/3106237.3117767>
- Amazon Web Services. (2025a). *Amazon SQS dead-letter queues*. Amazon SQS Developer Guide.
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>
- Amazon Web Services. (2025b). *Configuring event notifications using Amazon S3 event notifications*. Amazon S3 User Guide.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/NotificationHowTo.html>
- Amazon Web Services. (2025c). *Lambda function scaling*. AWS Lambda Developer Guide.
<https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>
- Amazon Web Services. (2025d). *Tracing user requests to REST APIs using X-Ray*. Amazon API Gateway Developer Guide.
<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-xray.html>
- Amazon Web Services. (2025e). *Using Amazon CloudWatch alarms*. Amazon CloudWatch User Guide.
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>
- Amazon Web Services. (2025f). *Using Lambda with Amazon SQS*. AWS Lambda Developer Guide. <https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html>

- Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V.,
Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J. E., Popa, R. A., Stoica, I., &
Patterson, D. A. (2019). *Cloud programming simplified: A Berkeley view on serverless
computing* (Technical Report No. UCB/EECS-2019-3). University of California,
Berkeley. <https://doi.org/10.48550/arXiv.1902.03383>
- Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly
Media.
- Open-Meteo. (2025). *Weather forecast API documentation*. <https://open-meteo.com/en/docs>
- Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.